

Heap

****Just design the function(s) specified in the problem.****

****No need to write driver code.****

****You may create helper functions to solve tasks****

****This document is a compilation of previous semester's exam (midterm/final) problems and lab quiz problems. Try to practice relevant problems from online judges as well.****

1.

Tasks from Lab 8 -  Lab 8 - Heap

2.

You are given an **array of integers** and a number **k**. Your task is to find the **sum of top k largest** elements from the list.

Note:

1. You need to figure out which heap to use.
2. You are not allowed to use any **direct sorting, searching on the array**.
3. You are not allowed to use any built-in functions except **len()**
4. You can assume **insert()** and **swim()** functions are **already implemented**. You **have to implement whatever else is needed**.

Sample Input:	Sample Output:
array = [11, 15, 8, 2, 31, 23] k = 3	sum = 69
Explanation:	
The array is [11, 15, 8, 2, 31, 23] and the top k largest elements are [31,23,15]. So the sum will be 31+23+15 = 69	

3.

You are given an **array of integers** and a **number k**. Your task is to find the **product of top k lowest** elements from the list.

Note:

1. You need to figure out which heap to use.
2. You are not allowed to use any **direct sorting, searching on the array**.
3. You are not allowed to use any built-in functions except **len()**
4. You can assume **insert()** and **swim()** functions are **already implemented**. You **have to implement whatever else is needed**.

Sample Input:	Sample Output:
array = [11, 15, 8, 2, 31, 23] k = 3	product = 176
Explanation:	
The array is [11, 15, 8, 2, 31, 23] and the top k lowest elements are [2, 8, 11]. So the product will be $2*8*11 = 176$	

4.

You are given several buckets with water levels represented by a min-heap. In each operation, you choose the smallest non-zero water level, denoted by x , and remove x units of water from all non-empty buckets. Your task is to determine the minimum number of operations required to empty all the buckets. You need to write the `extract_min()`, `sink()`, and the `minimum_operation_find()` methods /functions.

Sample Input:	Sample Output:
heap = [1, 5, 9, 10, 12]	5

Explanation			
Heap Array	Heap Visual	Heap Array	Heap Visual
Initial Stage: [1, 5, 9, 10, 12]	1 /\n 5 9 /\n 10 12	Step 3: Extracted the minimum value 4 [5, 7, 0, 0, 0] Subtracted the minimum value (4) from each non-zero value [1, 3, 0, 0, 0]	1 /\n 3 0 /\n 0 0
Step 1: Extracted the minimum value 1 [5, 9, 10, 12, 0] Subtracted the minimum value (1) from each non-zero value [4, 8, 9, 11, 0]	4 /\n 8 9 /\n 11 0	Step 4: Extracted the minimum value 1 [3, 0, 0, 0, 0] Subtracted the minimum value (1) from each non-zero value [2, 0, 0, 0, 0]	2 /\n 0 0 /\n 0 0
Step 2: Extracted the minimum value 4 [8, 9, 11, 0, 0] Subtracted the minimum value (4) from each non-zero value [4, 5, 7, 0, 0]	4 / \n 5 7 /\ /n 0 0 0	Step 5: Extracted the minimum value 2 [0, 0, 0, 0, 0]	0 /\n 0 0 /\n 0 0

5.

You are given several magical jars, each filled with a certain amount of liquid, represented by a min-heap. In each operation, the smallest non-zero liquid level, denoted by x , is extracted, and the liquid level in each non-empty jar is divided by x (using floor division). When a jar's liquid level becomes 0, it is considered empty. Your task is to determine the minimum number of operations required to completely empty all the jars.

You just need to write the `extract_min()`, `sink()`, and the `minimum_operation_find()` methods/functions.

Sample Input:	Sample Output:
heap = [2, 3, 5, 20, 100]	5

Explanation			
Heap Array	Heap Visual	Heap Array	Heap Visual
Initial Stage: [2, 3, 5, 20, 100]	2 / \ 3 5 / \ 20 100	Step 3: Extracted the minimum value 2. [10, 50, 0, 0, 0] Performed floor division by the minimum value 2. [5, 25, 0, 0, 0]	5 / \ 25 0 / \ 0 0
Step 1: Extracted the minimum value 2 [3, 5, 20, 100, 0] Performed floor division by the minimum value 2. [1, 2, 10, 50, 0]	1 / \ 2 10 / \ 50 0	Step 4: Extracted the minimum value 5. Performed floor division by the minimum value 5. [5, 0, 0, 0, 0]	5 / \ 0 0 / \ 0 0
Step 2: Extracted the minimum value 1 [2, 10, 50, 0, 0] Performed floor division by the minimum value 1. [2, 10, 50, 0, 0]	2 / \ 10 50 / \ 0 0	Step 5: Extracted the minimum value 5 [0, 0, 0, 0, 0]	0 / \ 0 0 / \ 0 0

6.

You are given two arrays A and B , both consisting of n integers. Starting from index 0, for each element in B , you must add it to the current smallest integer in A . After processing all elements in B , return array A sorted in non-decreasing order.

You are only allowed to use the `insert()` and `extract()` method from `MinHeap/MaxHeap`. Do not use any other functions (for example, `sort()`).

```
def add_arrays(n, A, B):  
    //to do
```

Input	Output	Explanation
$n = 3$ $A = [1, 3, 5]$ $B = [2, 4, 6]$	$A = [5, 7, 9]$	Initially, $A=[1, 3, 5]$, $B = [2, 4, 6]$ 1. Add $B[0]=2$ to the smallest integer $A[0]=1$. A becomes $[3, 3, 5]$. 2. Add $B[1]=4$ to the smallest integer $A[0]=3$. A becomes $[7, 3, 5]$. 3. Add $B[2]=6$ to the smallest integer $A[2]=3$. A becomes $[7, 9, 5]$. Finally, make A non-decreasing to get $[5,7,9]$.

7.

You are given two arrays A and B , both consisting of n integers. Starting from index 0, for each element in B , you must subtract it from the current largest integer in A . After processing all elements in B , return array A sorted in non-increasing order.

You are only allowed to use the `insert()` and `extract()` method from `MinHeap/MaxHeap`. Do not use any other functions (for example, `sort()`).

```
def sub_arrays(n, A, B):  
    //to do
```

Input	Output	Explanation
$n = 3$ $A = [1, 3, 5]$ $B = [2, 4, 6]$	$A = [1, -1, -3]$	Initially, $A=[1, 3, 5]$, $B = [2, 4, 6]$ 1. Sub $B[0]=2$ from the largest integer $A[2]=5$. A becomes $[1, 3, 3]$. 2. Sub $B[1]=4$ from the largest integer $A[2]=3$. A becomes $[1, 3, -1]$. 3. Sub $B[2]=6$ from the largest integer $A[1]=3$. A becomes $[1, -3, -1]$. Finally, make A non-increasing to get $[1, -1, -3]$.

8.

In applications like batch task scheduling, it's vital to manage tasks based on their urgency, represented by priority levels where a lower number signifies higher urgency. You are given two arrays: one for task IDs and the other for corresponding priorities. These tasks must be organized using a data structure designed for efficient access to the most urgent tasks. However, only those tasks within a specified priority range, marked by `lowPriority` and `highPriority`, should be considered. Tasks falling outside this range should be excluded.

[Note: The most urgent tasks have lower priority numbers, so the data structure should focus on retrieving the minimum values within the range to correctly identify and process the highest-priority tasks]

Note:

1. You cannot use any built-in functions except `len()`.
2. Direct sorting or searching methods on the array are not allowed.
3. Assume heap-related functions **`extractMin()`**, **`sink()`** are implemented and other necessary functions must be implemented.

Sample Input:	Sample Output:
<pre>tasks = [101, 102, 103, 104, 105, 106, 107, 108, 109] priorities = [9, 3, 7, 1, 5, 8, 2, 6, 4] lowPriority = 3 highPriority = 7</pre>	<pre>result = [102, 109, 105, 108, 103]</pre>
Explanation:	
Given <code>lowPriority = 3</code> and <code>highPriority = 6</code> , tasks with the third, fourth, fifth, and sixth highest priorities based on the priority scale are selected. This corresponds to the tasks with priorities 3, 4, 5, and 6 after selecting them by their priority, which are tasks 102, 105, 101, and 108.	

9.

In applications like task scheduling, it's vital to manage tasks based on their urgency, represented by priority levels where a higher number signifies higher urgency. You are given two arrays: one for task IDs and the other for corresponding priorities. These tasks must be organized using a data structure designed for efficient access to the most urgent tasks. However, only those tasks within a specified priority range, marked by `lowPriority` and `highPriority`, should be considered. Tasks falling outside this range should be excluded.

[Note: The most urgent tasks have higher priority numbers, so the data structure should focus on retrieving the maximum values within the range to correctly identify and process the highest-priority tasks]

Note:

1. You cannot use any built-in functions except `len()`.
2. Direct sorting or searching methods on the array are not allowed.
1. Assume heap-related functions **`extractMax()`**, **`sink()`** are implemented and other necessary functions must be implemented.

Sample Input:	Sample Output:
<pre>tasks = [101, 102, 103, 104, 105, 106, 107, 108, 109] priorities = [9, 3, 7, 1, 5, 8, 2, 6, 4] lowPriority = 3 highPriority = 7</pre>	<pre>result = [103, 108, 105, 109, 102]</pre>
Explanation:	
Given <code>lowPriority = 3</code> and <code>highPriority = 6</code> , tasks with the third, fourth, fifth, and sixth highest priorities based on the priority scale are selected. This corresponds to the tasks with priorities 7, 6, 5, and 4 after selecting them by their priority, which are tasks 103, 108, 105, and 109 respectively.	

10.

The Brac University Library is prioritizing books for purchase based on their **demand** and **ratings** by students. Two arrays are provided:

- **arr1**: Demand for books (number of requests).
- **arr2**: Average student ratings for the books (scale of 1 to 5).

Books with **higher priority scores** should be **purchased first**. The **purchase priority score** for each book is calculated as:

$$\text{priority_score} = \text{demand} \times \text{rating}$$

Your task is to:

1. Create the **purchase priority scores array** using the above formula.
2. Determine the **appropriate type of heap** required to prioritize books for purchase.
3. **Construct the heap** using the priority scores.
4. Extract the **top 3 books** to be purchased first based on their priority scores.

Sample Input:	Expected Output:
arr1 = [50, 80, 40, 60, 52] arr2 = [4.5, 4.8, 4.0, 4.2, 4.7]	Priority Scores: [225, 384, 160, 252, 245] Top Books: 384, 252, 245

Note: Assume the **extract()** and **sink()** methods are already implemented and can be used directly. You need to **implement other required methods** and **make the necessary method calls** to complete the task. You are not allowed to use any built-in functions except **len()**.

11.

Brac University's course rating system needs to prioritize courses based on **difficulty** and **student satisfaction**. Two arrays are provided:

- **arr1**: Difficulty levels (1 to 10, higher values = more difficult).
- **arr2**: Student satisfaction scores (1 to 10, higher values = more satisfied).

Courses with **lower overall ratings** should be **prioritized first** for improvement. The **overall rating** for each course is calculated as:

$$\text{overall_rating} = \text{difficulty level} \times \text{satisfaction score}$$

Your task is to:

1. Create the **overall ratings array** using the above formula.
2. Determine the **appropriate type of heap** needed to prioritize courses based on the task description.
3. **Construct the heap** using the overall ratings.
4. Extract the **top 3 courses for improvement** based on their overall ratings.

Sample Input:	Expected Output:
arr1 = [7, 6, 5, 9, 8] arr2 = [6, 8, 7, 5, 4]	Overall Ratings: [42, 48, 35, 45, 32] Top Courses: 32, 35, 42

Note: Assume the **insert()** and **swim()** methods are already implemented and can be used directly. You need to **implement other required methods** and **make the necessary method calls** to complete the task. You are not allowed to use any built-in functions except **len()**.

12.

You are given an **array of integers** and values of `x` and `y`. You need to create a new heap consisting of the numbers between x^{th} lowest position and y^{th} lowest position from the array.

Note:

1. You need to figure out which heap to use.
2. You are not allowed to use any **direct sorting, searching on the array**.
3. You are not allowed to use any built-in functions except `len()`
4. You can assume `insert()` and `swim()` functions are **already implemented**.

You have to implement whatever else is needed.

Sample Input:	Sample Output:
array = [11, 15, 8, 2, 31, 23] x = 2 y = 5	result = [8, 11, 15, 23]
Explanation:	
When the array is sorted [2, 8, 11, 15, 23, 31] the values are in lowest to highest order. For each position of n we can say, If:- n = 1, Position-1 : 2 (1-st lowest) n = 2, Position-2: 8 (2-nd lowest) n = 5, Position-5: 23 (5-th lowest) n = 6, Position-6: 31 (6-th lowest)	Here, x = 2, 2-nd lowest: 8 y = 5, 5-th lowest: 23 Numbers between 2nd and 5th lowest in the given array are: 8, 11, 15, 23. So the new heap will have these values.

13.

You are given an **array of integers** and values of `x` and `y`. You need to create a new heap consisting of the numbers between x^{th} highest position and y^{th} highest position from the array.

Note:

1. You need to figure out which heap to use.
2. You are not allowed to use any **direct sorting, searching on the array**.
3. You are not allowed to use any built-in functions except `len()`
4. You can assume `insert()` and `swim()` functions are **already implemented**. You have to implement whatever else is needed.

Sample Input:	Sample Output:
<code>array = [11, 15, 8, 2, 31, 23]</code> <code>x = 2</code> <code>y = 5</code>	<code>result = [23, 15, 11, 8]</code>
Explanation:	
When the array is sorted [31, 23, 15, 11, 8, 2] the values are in highest to lowest order For each position of n we can say, If:- n = 1 (Position-1) , 1-st highest: 31 n = 2 (Position-2) , 2-nd highest: 23 n = 5 (Position-5) , 5-th highest: 8 n = 6 (Position-6) , 6-th highest: 2	Here, x = 2, 2-nd highest: 23 y = 5, 5-th highest: 8 Numbers between 2nd and 5th highest in the given array are: 23, 15, 11, 8 So the new heap will have these values.

14.

As a Software Engineer at Microsoft, you need to schedule tasks on the CPU based on priority. Given n tasks with unique priorities in an array and a value k , perform the first k tasks with the highest priority.

Implement the function `cpu_scheduler(tasks, k)` to print the task numbers of the top k tasks in descending order of priority. Use the provided `MaxHeap` or `MinHeap` classes with `insert(item)` and `extract()` (extract will return the extracted value) methods. **You cannot iterate through the array manually to find out the highest priority task. Check sample input-output to find out the printing format.**

You can create an Empty Heap using the following code	
Python Notation	Java Notation
<pre>max_heap = MaxHeap() min_heap = MinHeap()</pre>	<pre>MaxHeap maxHeap = new MaxHeap() MinHeap minHeap = new MinHeap()</pre>

Python Notation	Java Notation
<pre>def cpu_scheduler(tasks, k): #To do</pre>	<pre>static void cpu_scheduler(int[] tasks, int k){ //To do }</pre>

Input	Output	Explanation
<pre>tasks = [45, 70, 85, 60, 90, 75] k = 3</pre>	<pre>Task 1 - Priority 90 Task 2 - Priority 85 Task 3 - Priority 75</pre>	Here are the 6 tasks given in the array. The value of k is 3 so the function will print the first 3 tasks with higher priority according to the shown format.

15.

You are given an array A of size N. You can perform an operation in which you will remove the largest and the smallest elements from the array and add their difference back into the array. You are given an integer k that will define the number of times you have to perform the operation. Write a method heapSum() that takes an array, an integer k as parameters, performs k operations on the array, and returns the sum of the array elements after the operations.

You are given MaxHeap and MinHeap classes. The methods in the MaxHeap class are - MaxInsert(), MaxDelete() and the methods in the MinHeap class are - MinInsert(), MinDelete(). The constructor of both classes does not take any parameters. The method works according to their names and no need to implement any of the methods.

YOU MUST USE THE GIVEN HEAP DATA STRUCTURES TO SOLVE THIS PROBLEM.

Hint: No need to modify the given array. Remember that the underlying data structure of a heap is an array.

Python Notation:
def heapSum(A, k):
 # To Do

Sample Input	Sample Output	Explanation
heapSum(A, k) A=[3, 2, 1, 5, 4,] k = 2	9	After 1st operation, the array will become, A = [3,2,4,4]. After the 2nd operation, the array will become, A = [3,2,4]. sum of elements = 9. Here the shown value of A is just to explain the output and does not mean the state of the given array after each operation

16.

- 1) Insert 10, 5, 20, 15, 17 one after another in a **max-heap** and draw the **max-heap** after each insertion.

17.

Grace Marks to Rescue 🚒

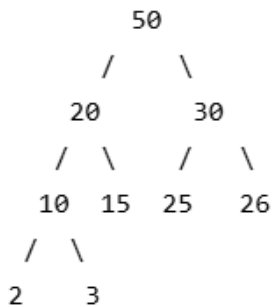
A teacher uses an automated grading system that stores students' exam scores in a min-heap (*with the lowest score at the top*) for efficient processing. Before publishing the ranking from highest to lowest, the teacher decides to **add one grace mark** to every student's score (*capping the maximum at 100*) and then convert the updated scores from **a min-heap to a max-heap** so the highest score is at the top.

Now, write a function, **rescue(n, scores)**, that takes n as the number of students' scores in the list and *scores* as a list of n integers representing the students' scores in min-heap order.

Sample Input	Sample Output	Hint
n = 5 scores = [56, 59, 69, 89, 100]	[100, 90, 70, 60, 57]	Implement your own <i>heapify</i> function and conversion logic to build a <i>max-heap</i> . Do not create a new array for the <i>max-heap</i> . Perform the heap conversion in-place . Your function should return the <i>max-heap</i> array after conversion, not print it.

[You are not allowed to use any built-in functions and heap libraries]

18.



2. From the given Max Heap tree, first remove 20 and
3. Then add 12 in that tree. Draw the final executed Max Heap tree. [2 points]

Answer:

19.

5. If a Max heap is constructed with 45 nodes, how many leaf nodes will it contain? [1 point]

20.

In an F1 qualifying session, each driver completes two laps.
You are given three arrays:

drivers → the names of drivers

lap1[] → time taken in the first lap

lap2[] → time taken in the second lap

A driver's best lap time (minimum of lap1 and lap2) is considered their priority.
Smaller lap time = higher priority

You must organize the drivers using the appropriate heap so that they are ranked from fastest to slowest, based on their best lap times.

Constraints:

- You cannot use any built-in functions except len().
- Direct sorting or searching methods on the array are not allowed.
- Assume a heap class is created and heap-related functions **extractMin()**, **extractMax()**, **sink()**, **swim()** are implemented and can be used while other necessary functions must be implemented. Assume the heap class has two arrays to store the best lap and names.

Sample Input:	Sample Output:
drivers = ["Verstappen", "Hamilton", "Leclerc", "Norris", "Russell"] lap1 = [75, 78, 77, 79, 80] lap2 = [76, 74, 78, 80, 79]	Result = ["Hamilton", "Verstappen", "Leclerc", "Norris", "Russell"]
Explanation:	
Each driver has two lap times, and their best lap (the smaller time) determines their ranking. Faster drivers have lower lap times, so we rank them from fastest to slowest based on their best lap. For example, Hamilton's best lap is 74 seconds and Verstappen's is 75, so Hamilton is ranked higher.	

21.

You are given the following numbers to insert one by one into a **min-heap**:

[10, 4, 15, 2, 7]

Show the state of the heap **after each insertion** (as a binary tree), clearly indicating the changes at every step.

22.

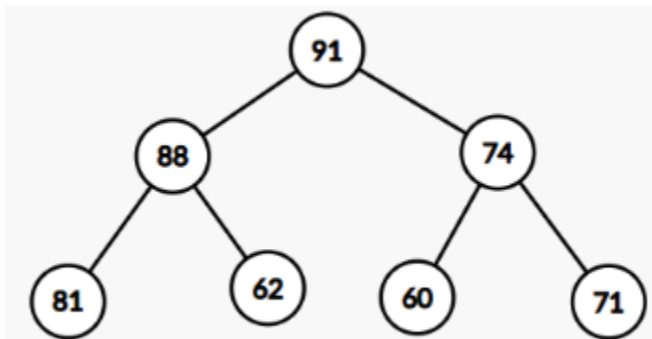
You are given an array of integers and a value k . Your task is to find the modulus of top n elements from the list, where n is the number of i -th elements you have to mod with the remainder from previous modulus operation, **only if the remainder is odd, otherwise just mod with double the remainder**. Initially divide the largest element by k to get the first remainder and if the remainder is 0 for noninitial cases, divide by k . You have to find the final result. [10 marks]

Note:

1. You need to figure out which heap to use.
2. You are not allowed to use any direct sorting, searching on the array.
3. You are not allowed to use any built-in functions except `len()`.
4. You have to implement the methods you require.

Sample Input:	Sample Output:
<code>arr = [15, 22, 7, 30, 18]</code> <code>k = 5</code>	Result = 1
Explanation:	
1. $30 \% 5 = 0 \rightarrow \text{remainder} = 0$ (even). - o Next divisor = double remainder = 0 $\rightarrow$ but modulus by 0 invalid. - o To handle this case, we assume: if divisor = 0, continue using original k. 2. So next divisor = 5. 3. $22 \% 5 = 2 \rightarrow \text{remainder} = 2$ (even). - o Next divisor = $2 \times 2 = 4$. 4. $18 \% 4 = 2 \rightarrow \text{remainder} = 2$ (even). - o Next divisor = $2 \times 2 = 4$. 5. $15 \% 4 = 3 \rightarrow \text{remainder} = 3$ (odd). - o Next divisor = 3. 6. $7 \% 3 = 1 \rightarrow \text{remainder} = 1$ (odd). - o Next divisor = 1 (but no more elements left).	

23.



Draw the heap after 2 `extractMax()` calls

24.

A sports academy is compiling the performance scores of its athletes. Due to a sensor malfunction, some scores have been mistakenly recorded as negative numbers. Since a score cannot be negative, these values should be treated as their absolute values. Your task is to write a function `sortScores()` that takes an array of integers representing the athlete's scores. Then sort the scores in descending order and return the modified sorted array.

- The solution must not exceed $O(n \cdot \log(n))$ time complexity.
- You can use **MinHeap** data structure.
- You can not take additional arrays or other data structures except Heap.

The **MinHeap** class has already been defined for you. You can create an instance of this class and use the public functions `insert()` and `extract()`. When creating the heap, you will need to specify its capacity. *No need to write the MinHeap class on your own.*

Sample Input	Sample Output	Explanation
[50, -20, 75, -10, 90, 30]	[90, 75, 50, 30, 20, 10]	All the numbers are treated as absolute values and then sorted in-place using a MinHeap.
[40, -40, 25, -25, 40, 10]	[40, 40, 40, 25, 25, 10]	

//java function signature

```
public static int[] sortScores(int[] scores){
```

#python function signature

```
|| def sortScores(scores):
```

25.

You are given an array of tasks, where each element represents the **urgency level** of that task and a deadline (number of days available). Your job is to complete the possible tasks within that deadline. However, there are a few rules:

- Each task takes **1 day** to complete.
- At each step, complete the task with the **highest urgency**.
- Ignore tasks left after the deadline.

The time complexity of your solution must be **less than $O(n^2)$** . Assume that the classes **MaxHeap** and **MinHeap** are provided with all necessary methods, including `insert(value)` and `extract()`, where `extract()` returns the removed element.

Caution: Using a Nested loop will result in $O(n^2)$ time complexity



You can create an Empty Heap using the following code	
Python Notation	Java Notation
<pre>max_heap = MaxHeap() min_heap = MinHeap()</pre>	<pre>MaxHeap maxHeap = new MaxHeap() MinHeap minHeap = new MinHeap()</pre>

Python Notation	Java Notation
<pre>def urgent_task(tasks, deadline): #To do</pre>	<pre>static void urgent_task(int[] tasks,int deadline){ //To do }</pre>

Input	Output	Explanation
<pre>tasks = [30, 95, 80, 25, 50] deadline = 3</pre>	<pre>Day 1 - Urgency 95 Day 2 - Urgency 80 Day 3 - Urgency 50 Tasks completed: 3 Tasks ignored: 2</pre>	Here, the deadline is 3 days and tasks 2, 3 and 5 have the highest urgency. So, they will be done in their respective urgency order. The rest of the tasks will be ignored.

26.

Complete the function below that will take an array of tasks and print out the completion step of tasks based on their priorities. The value of the index of an array will represent the priority of that task. The lower the value, the higher the priority. One task will be done in one step. **Assume that all the values of the array will be unique, and you have a magical function `getIndex(arr, val)` which will return the index of that value in the array in constant time.**

The time complexity of your solution must be less than $O(n^2)$. You may use the provided `MaxHeap` or `MinHeap` classes with the `insert(item)` and `extract()` (extract will return the extracted value) methods. **Check the sample input-output to find out the printing format.**

You can create an Empty Heap using the following code	
Python Notation	Java Notation
<pre>max_heap = MaxHeap() min_heap = MinHeap()</pre>	<pre>MaxHeap maxHeap = new MaxHeap() MinHeap minHeap = new MinHeap()</pre>

Python Notation	Java Notation
<pre>def priority_task(tasks): #To do</pre>	<pre>static void priority_task(int[] tasks){ //To do }</pre>

Input	Output	Explanation
<pre>tasks = [60, 85, 70, 45]</pre>	<pre>Step 1 - Task 4 Step 2 - Task 1 Step 3 - Task 3 Step 4 - Task 2</pre>	Here, index 3 has the minimum value 45, so it will be done at step 1. <code>getIndex(tasks, 45)</code> will return 3, hence the task number is $(3 + 1) = 4$. The rest of the output is generated following the same pattern.